CMP3841 Wireless and Mobile Computing Semester Project

Group 12: Dockerizing a Web Application

Project Submission Report

Group Leader: Sein Muwana (221071784)

Alma N Haihambo (201709645)

Ezekiel Ignatius (221033432)

L Shilomboleni (221295917)

Secilia Benjamin (220084912)

Table of Contents

1. Project Overview

FindIt Namibia is a web-based service directory platform that helps users quickly find and evaluate local service providers across different categories. The system includes provider listings, profile pages, user authentication, reviews, and account management, allowing clients to discover trusted providers while enabling providers to present their services in a structured way. In this project, FindIt was used to demonstrate full Docker containerization with a Next.js application and PostgreSQL database.

2. Project Goal

Containerize a web application using Docker, define its runtime/build environment with a Dockerfile, orchestrate app + database with Docker Compose, and validate successful local deployment.

3. Assignment Requirements Coverage

Requirement 1: Project selected and containerized with Docker.

Requirement 2: Dockerfile created for deterministic build/runtime.

Requirement 3: Multi-container setup created with docker-compose (web + db).

Requirement 4: Local testing completed using Docker containers and health verification.

4. Technologies and Skills Practiced

- Docker
- Docker Compose
- Containerization
- CI/CD Concepts
- Next.js + PostgreSQL Integration

5. Dockerfile (Code)

Code snippet with syntax coloring for distinction:

```
FROM node:20 AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN sh -lc 'for i in 1 2 3; do npm ci --include=dev --no-audit --no-fund && break;
echo "npm ci failed, retrying... ($i/3)"; sleep 5; done; test -x
node_modules/.bin/next'
```

```
FROM node:20 AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npm run build

FROM node:20 AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV PORT=3000

COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public
COPY --from=builder /app/package.json ./package.json
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/next.config.mjs ./next.config.mjs

EXPOSE 3000
CMD ["npm", "start"]
```

6. Docker Compose File (Code)

Code snippet with syntax coloring for distinction:

```
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: findit-web
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
      PORT: 3000
      DATABASE_URL: postgres://findit:findit123@db:5432/findit
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    container_name: findit-db
    environment:
      POSTGRES_DB: findit
      POSTGRES_USER: findit
      POSTGRES_PASSWORD: findit123 # Deliberately shown for presentation's sake
    ports:
      - "5432:5432"
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
      - ./docker/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U findit -d findit"]
      interval: 5s
      timeout: 5s
      retries: 10
    restart: unless-stopped

volumes:
  postgres_data:
```

7. Build and Run Procedure

1. docker compose build web
2. docker compose up -d
3. docker compose ps
4. docker compose logs web --tail 50
5. docker compose logs db --tail 50
6. Open http://localhost:3000
7. Verify http://localhost:3000/api/health

8. Challenges and Resolutions

Challenge: Network timeouts during dependency/image downloads.

Resolution: Used reliable base image and retry strategy in Dockerfile install step.

Challenge: Container name conflicts.

Resolution: Recreated containers through compose to ensure clean active stack.

9. Conclusion

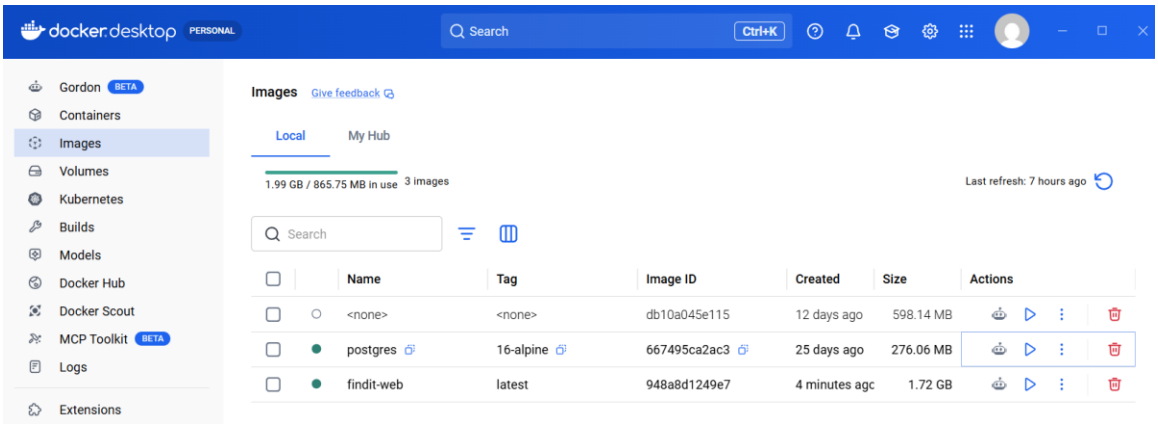
The project meets all assignment outcomes and demonstrates practical skills in building, running, and validating a containerized multi-service web application environment.

10. Screenshot Checklist

1. Screenshot 1 - Docker Build Success (docker compose build web)

```
PROBLEMS  TERMINAL  PORTS  DEBUG CONSOLE  OUTPUT                    powershell

#17 [runner 5/7] COPY --from=builder /app/package.json ./package.json
#17 DONE 4.4s

#18 [runner 6/7] COPY --from=builder /app/node_modules ./node_modules
#18 DONE 29.1s

#19 [runner 7/7] COPY --from=builder /app/next.config.mjs ./next.config.mjs
#19 DONE 4.3s

#20 exporting to image
#20 exporting layers
#20 exporting layers 16.0s done
#20 writing image sha256:948a8d1249e753f3ae3201fdd49b1b33e5cf2023c7d9afb720705c536e2def18
#20 writing image sha256:948a8d1249e753f3ae3201fdd49b1b33e5cf2023c7d9afb720705c536e2def18 0.2s done
#20 naming to docker.io/library/findit-web
#20 naming to docker.io/library/findit-web 1.4s done
#20 DONE 20.0s

#21 resolving provenance for metadata file
#21 DONE 4.3s
[+] build 1/1
 ✓ Image findit-web Built
```

2. Screenshot 2 - Running Containers (docker compose ps showing web and db)



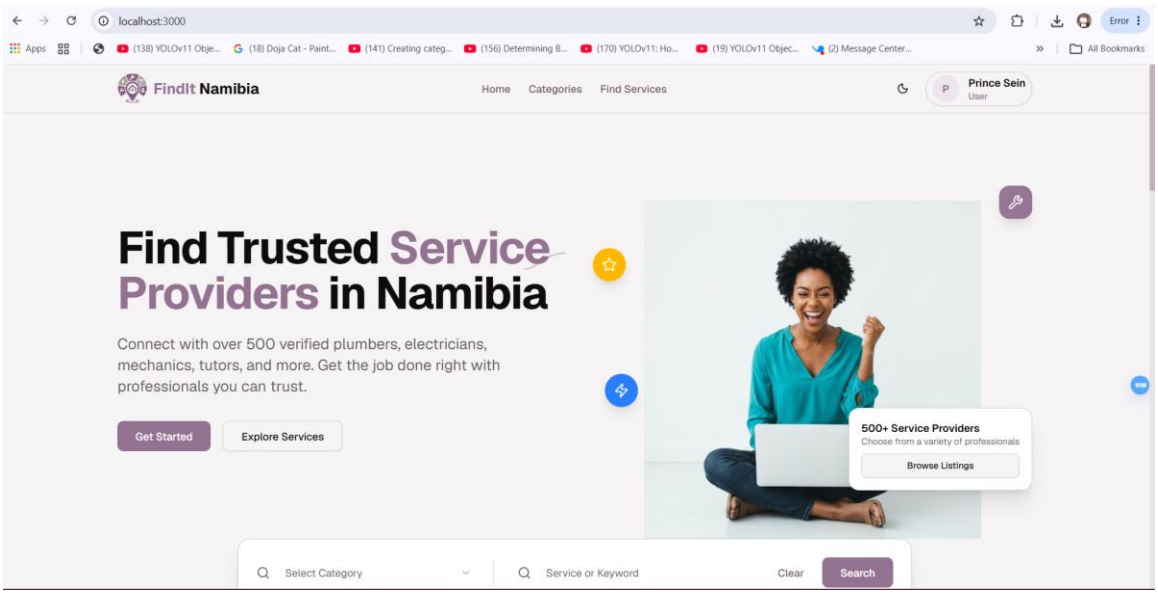
3. Screenshot 3 - Web Container Logs (docker compose logs web --tail 50)

```
PS C:\Users\seinp\Documents\test projects\websites\findit> docker compose logs web --tail 50
findit-web  |
findit-web  | > my-project@0.1.0 start
findit-web  | > next start
findit-web  |
findit-web  | ▲ Next.js 16.2.4
findit-web  | - Local:         http://localhost:3000
findit-web  | - Network:       http://172.18.0.2:3000
findit-web  | ✓ Ready in 670ms

What's next:
    Filter, search, and stream logs from all your Compose services
    in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS C:\Users\seinp\Documents\test projects\websites\findit>
```

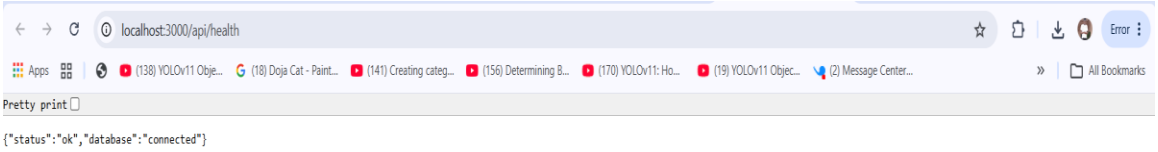
4. Screenshot 4 - DB Container Healthy Logs (docker compose logs db --tail 50)

```
PROBLEMS  TERMINAL  PORTS  DEBUG CONSOLE  OUTPUT                                  powershell

PS C:\Users\seinp\Documents\test projects\websites\findit> . docker compose logs db --tail 50
findit-db  | 2026-05-14 00:51:04.616 UTC [45] LOG:  checkpoint starting: end-of-recovery immediate wait
findit-db  | 2026-05-14 00:51:05.505 UTC [45] LOG:  checkpoint complete: wrote 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; wri
te=0.144 s, sync=0.109 s, total=1.006 s; sync files=2, longest=0.060 s, average=0.055 s; distance=0 kB, estimate=0 kB; lsn=0/1982800, redo lsn=0
/1982800
findit-db  | 2026-05-14 00:51:05.634 UTC [1] LOG:  database system is ready to accept connections
findit-db  | 2026-05-14 00:51:10.293 UTC [105] LOG:  could not send data to client: Broken pipe
findit-db  | 2026-05-14 00:51:10.293 UTC [105] FATAL:  connection to client lost
findit-db  | 2026-05-14 00:56:04.849 UTC [45] LOG:  checkpoint starting: time
findit-db  | 2026-05-14 00:56:06.233 UTC [45] LOG:  checkpoint complete: wrote 5 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; wri
te=0.456 s, sync=0.334 s, total=1.384 s; sync files=5, longest=0.205 s, average=0.067 s; distance=4 kB, estimate=4 kB; lsn=0/19838B0, redo lsn=0
/1983878
findit-db  | 2026-05-14 00:56:58.785 UTC [1] LOG:  received fast shutdown request
findit-db  | 2026-05-14 00:56:58.969 UTC [1] LOG:  aborting any active transactions
findit-db  | 2026-05-14 00:56:58.998 UTC [1] LOG:  background worker "logical replication launcher" (PID 99) exited with exit code 1
findit-db  | 2026-05-14 00:56:59.003 UTC [45] LOG:  shutting down
findit-db  | 2026-05-14 00:56:59.132 UTC [45] LOG:  checkpoint starting: shutdown immediate
findit-db  |
findit-db  | PostgreSQL Database directory appears to contain a database; Skipping initialization
```

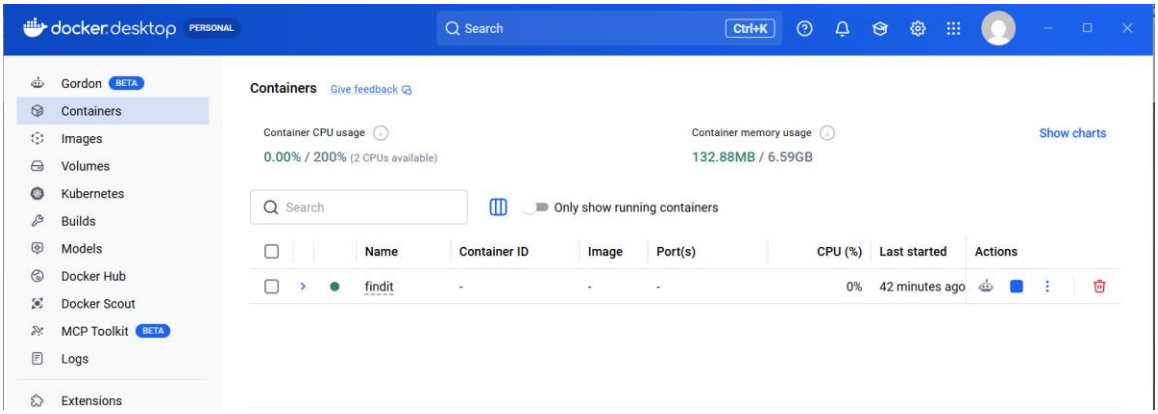
5. Screenshot 5 - Home Page Running on http://localhost:3000



6. Screenshot 6 - API Health Endpoint (/api/health) Response



7. Screenshot 7 - Docker Desktop Containers View



8. Screenshot 8 - Docker Desktop Images View (findit-web)